SafeCharge-RL

Reinforcement learning for EV charging when drivers' departure times are wrong or misreported

Presentation content for the mid-review: 15 minutes, 9 slides, 3 speakers. Slide text is kept very short; the right column is what to say.

Team 32	Adithyan R (AM.SC.U4AIE23065), Adithya Rajesh (AM.SC.U4AIE23007), Balu M Krishna (AM.SC.U4AIE23026)
Guide	Dr. Geetha M, Dept. of CSE (AI), Amrita Vishwa Vidyapeetham
Prepared	9 Oct 2026, for the mid-review (9-10 Oct). Code: github.com/adithyx-0/safecharge-rl (private repo)

Time plan (15 minutes in total)

Slide	Topic	Time	Speaker
1	Title and team	0:30	Member 1
2	Problem and why RL	1:30	Member 1
3	Existing work and gap	1:30	Member 1
4	MDP design	2:00	Member 2
5	Algorithm and data	1:30	Member 2
6	System design diagram	1:00	Member 2
7	Novel components	1:00	Member 2
8	Results so far	2:00	Member 3
9	Live demo + next plan	3:00	Member 3
	Buffer	1:00	

Totals: Member 1 3:30, Member 2 6:00, Member 3 5:00, buffer 1:00. If you run long, cut the demo to 1 minute (the table only). Swap the member numbers to fit who is comfortable with what. Charts to drop into slides are in `docs/briefing/fig/`: e1.png, e2.png, e3.png, train.png, demo_day.png; the system figure is `docs/architecture_diagram.jpg`.

Rules for the talk

- One idea per slide; read nothing word for word. Numbers: say at most two per slide.
- Do not say 'we beat the baselines'. We do not. Say what we found: ties EDF on cost, charges gently, but can be gamed.
- Say 'we simulate misreporting; we do not claim how often real drivers lie'.
- If asked about progress: 'core implementation and four of six experiments are done; the rest is running or planned'. Do not quote 80%.

Slides

Slide 1: Title and Team (0:30, Member 1)

On the slide (keep it this short)	What to say
- SafeCharge-RL: Reinforcement Learning for EV Smart Charging - Team 32: Adithyan R, Adithya Rajesh, Balu M Krishna - Guide: Dr. Geetha M - RL area: constrained RL for sequential decision-making	We decide how much power each car gets at a shared charging station, every 15 minutes. Our twist: the departure time the driver reports can be wrong or faked.

Show: Title, team names, guide, Amrita logo.

Slide 2: Problem Background and Statement (1:30, Member 1)

On the slide (keep it this short)	What to say
- Many EVs, one transformer: total power under a grid limit - Goals pull apart: cost, deadlines, battery wear - Scheduler sees only the REPORTED departure time; reports are often wrong or faked - Why RL: unknown future, sequential decisions - Problem: stay under the cap, serve cars, cut cost, survive bad reports	Think of a workplace car park behind one transformer. Cars come and go; electricity is cheap at night and expensive in the afternoon. The system decides who charges now. It cannot see the future, and it only knows what drivers typed in. RL fits because decisions are sequential and the future is unknown; exact optimisation needs the whole day in advance. Our research question: what happens to a learned policy when reports are wrong, and can a driver cheat it?

Show: A simple picture: cars, one grid limit, price curve. Optional: the price chart (cheap 0-8 h, peak 12-18 h).

Slide 3: Existing Work, Gap and Our Difference (1:30, Member 1)

On the slide (keep it this short)	What to say
- Li, Wan, He 2020: safe RL, one charger, no shared cap - Chen, Shi 2022: Lagrangian + SAC, one EV, synthetic data - Ting et al. 2024: station limit, early departures as random noise, simulated EVs - Li, Sun 2023: real Caltech data, shows user-entered times are unreliable - Ferragut et al. 2021: under-reporting gains service, but only for rule-based policies - GAP: no study of a LEARNED policy that drivers can GAME, on REAL sessions with a shared hard cap, vs hindsight optimum	Safe and constrained RL for charging exists, but for single chargers or simulated cars. Departure uncertainty is treated as random noise. Ferragut showed that for simple rules a driver can gain by reporting early. To our search, nobody checked whether a learned policy has the same weakness on real data. That is our focus. We do not claim a new RL algorithm.

Show: Table: paper, what it does, limit. Last row highlighted: our difference.

Slide 4: MDP Design (2:00, Member 2)

On the slide (keep it this short)	What to say
- Agent: station controller; Environment: simulator on real days - State: per port energy left, reported time left, laxity; price, time of day - Action: 0-1 of max power per port, then safety cut to the cap - Reward: minus electricity cost; Costs: unmet energy, wear proxy - Episode: one weekday, 96 steps; true departure hidden	Each day is one episode. At every step the agent sees the cars plugged in with remaining energy and the reported time left. It proposes a power fraction per port. A safety projection reduces the proposal so the total stays within the cap: that is a hard guarantee. Reward is the negative bill. Two extra costs have budgets: unmet energy, and a wear proxy because we have no battery data. Cars always leave at their true time, so an early report can cause unmet energy.

Show: The MDP table. If time allows, the 3-line projection example (20 kW cap, asks 12/10/8, each cut by 3.33).

Slide 5: RL Algorithm and Data (1:30, Member 2)

On the slide (keep it this short)	What to say
- PPO-Lagrangian: PPO + a multiplier per cost - Deep Sets network: any number of cars - Data: real ACN-Data, Caltech 31,424 / JPL 33,638 sessions - Train Jan-Aug 2019, test Nov 2019-Feb 2020 (85 days) - Our assumptions: summer tariff, grid cap = 20% of peak	PPO is a standard stable method. The multipliers turn 'stay within budget' into a trade-off the policy learns. Deep Sets shares one small network across ports, so it handles any number of cars. Data are real charging sessions, downloaded through the ACN API and checked against its totals. Price and the grid cap are our assumptions because the data have neither. Honest note: we added a behaviour-cloning warm start from a rule, otherwise learning was too slow. This is not in the submitted design.

Show: Small diagram of the network (port encoder, pool, head) is optional. Data timeline bar: train / val / test.

Slide 6: System Design Diagram (1:00, Member 2)

On the slide (keep it this short)	What to say
- Real sessions, report model, environment, RL agent, safety projection, back to environment	Left to right: real sessions go through the report model, which makes the reported time truthful or misreported. The environment tracks ports, price and the grid limit. The agent proposes power, the projection enforces the cap, the applied power moves the environment forward and returns state and reward.

Show: docs/architecture_diagram.jpg. Point at box 2 (our novelty) and box 5 (the guarantee).

Slide 7: Novel Components (1:00, Member 2)

On the slide (keep it this short)	What to say
- Reported departure treated as manipulable input to a LEARNED policy, on real data - Paired-replay measure of misreporting gain - Hard projection + Lagrangian budgets for a shared multi-car station - Comparison with hindsight-optimal schedule on real sessions - Test of the Ferragut fix (serve until declared departure) on RL - Extra finding: real driver reports are common and 2-3 h wrong	The new part is not the algorithm but the question and the evaluation: can a driver game a learned policy? We measure it with paired replay. We also found that the submitted claim 'reports are rare' does not hold on the full data: 52% of Caltech and 94% of JPL sessions have one.

Show: List with the first two items highlighted.

Slide 8: Results So Far (2:00, Member 3)

On the slide (keep it this short)	What to say
- E1: ties EDF on cost; charges gently (high-power ratio 0.17 vs 0.77) - E2: wrong reports hurt: learned 2.6x, rules 1.2-1.6x, MPC 30x - E3: lying early gains energy in 93-97% of cases (learned policy) - E4: serve-until-declared-departure removes the gain - E6: fine post-COVID, fails on JPL	Under truthful reports the learned policy is not better than simple rules on cost; its strength is gentle charging at similar cost. With wrong reports it degrades more than the rules, but far less than MPC. Our main finding: it can be gamed. The gain is small in kWh, but it happens almost every time. A simple fix, stopping service at the declared departure, removes the incentive, at the price of hurting honest early reports. Every number comes from a script and config with 5 seeds, 85 test days, 95% confidence intervals; the grid limit was never violated.

Show: docs/briefing/fig/e1.png and e3.png (E1 table is in the appendix on the last page). Keep the E3 chart visible while you speak: it is the central claim.

Slide 9: Demo So Far and Next Plan (3:00, Member 3)

On the slide (keep it this short)	What to say
- Live demo: busy day, all methods, no grid violation - Cheat test on a crowded day - Done: simulator, safety layer, baselines, PPO-Lag, E1-E3, E6, 108 tests - Next: finish E4, E5 ablation, unprojected baseline, report	(2 min demo) Here is one real busy day. The learned policy and EDF both hit the 34.6 kW cap and never exceed it. Notice the learned policy runs cars at lower power. Now a crowded day: three cars report one hour early. Rules give extra energy to some, the learned policy to more. (1 min) The implementation is complete up to the first four experiments and is on GitHub. What remains is finishing the E4 variants, the projection ablation and the report. We are confident the plan is achievable.

Show: Terminal: scripts/demo.py 58 --plot, then scripts/demo.py 7. Fallback if the live run fails: screenshot of the table and docs/briefing/fig/demo_day.png and train.png.

Questions the panel may ask (concept level)

Question	Short answer
Why RL and not optimisation?	The future is unknown and the reports are wrong. Exact optimisation needs the whole day; MPC is good with true reports but collapses (about 30x more unmet energy) with wrong ones.
Is the safety guaranteed?	The grid cap yes (projection inside the environment, 0 violations). Deadlines are within a budget, not guaranteed.
Why does the learned policy not beat the rules?	Charging cost differences are small; rules already charge fast and early. Our contribution is the robustness study, not a new best scheduler.
Do real drivers misreport?	We do not claim that. Recorded reports are often wrong (about 3 h), and we simulate deliberate early reports to test exploitability.
What is your novelty then?	Treating the report as a manipulable input of a learned policy, the paired-replay gain measure, and testing the Ferragut fix on RL, all on real sessions.
Why did JPL fail?	The policy was trained on Caltech's smaller sessions and does not transfer; we report this as a negative result.
What is in the code beyond the design?	Behaviour-cloning warm start, gamma 0.999, dual clipping, our own simulator and PPO-Lagrangian. All disclosed in the project log.
How far along are you?	Core implementation complete; four of six experiments done, E4 running; E5 and optional SAC remain.

Things to avoid: quoting results that are not in results/tables, saying the policy 'solves' the problem, claiming real drivers cheat, or saying the project is 100% done.

Appendix: E1 table (test days, truthful reports) for slide 8

Method	Cost $/kWh	Gap to optimum	Unmet energy	Missed-deadline vehicles	High-power ratio
Offline optimum	0.1564	-	0.43%	0.6%	0.79
MPC	0.1649	+5.5%	0.67%	2.5%	0.77
FCFS	0.1708	+10.2%	4.79%	9.1%	0.78
Learned (PPO-Lag)	0.1728	+11.5%	1.32%	7.4%	0.17
EDF	0.1732	+11.7%	1.25%	1.2%	0.77
LLF	0.1738	+12.1%	0.43%	1.7%	0.77

Gap = per-day cost gap to the offline optimum. High-power ratio = share of charging port-steps above 80% of max power (wear proxy).